QUALITY ASSURANCE TESTING DOCUMENTATION (QATD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

Table of Content

Document Control **3**

PRELIMINARY ROUND (Test Strategy & Planning) **3**

1. Scope & Requirements Traceability 3
2. Risk Assessment & Mitigation Strategy 3
3. Test Environment & Execution Strategy 5
4. CI/CD Release Thresholds & Automation Gates 5
5. Test Case Specifications (Drafts) 6
6. Test Strategy & Plan Sign-Off 7

© UMHackathon 2026

Email: umhackathon@um.edu.my

Document Control

Field	Detail
System Under Test (SUT)	*[e.g., UMHackathon 2026 - Team **YFood**]*
Team Repo URL	*[Insert link to your GitHub/GitLab here]*
Project Board URL	*[Insert link to your Jira/Trello Projects board]*
Live Deployment URL (optional for preliminary round)	*[Insert link to the live hosted application]*

Objective:

*[e.g., The primary objective is to ensure that **YFood** can handle food ordering, processing of payments, notify the vendors and assignment of riders reliably in addition to concurrency of traffic, with CI/CD checkpoints.]*

PRELIMINARY ROUND (Test Strategy & Planning)

1. Scope & Requirements Traceability

*This section aligns the testing connected back to specific user requirements via **Requirement Traceability Matrix".** So, it ensures every test has been conducted to prevent bugs/failure in products for that specific requirements and unplanned feature creep.*

1.1 In-Scope Core Features

- **Food Ordering:** Browsing Menu, adding items to cart, confirm order
- **Payment Processing:** Payment through card, e-wallet and generate receipts with history
- **Vendors & Delivery:** Notify the vendors, assignment of riders, and tracking.

1.2 Out-of-Scope

- Promotions & Offers
- Profile Management: Edit profile info, Delivery Address, Pictures
- App Theme

© UMHackathon 2026

Email: umhackathon@um.edu.my

2. Risk Assessment & Mitigation Strategy

*[e.g. Quality Assurance risks are anticipatory. So, identify the technical risk associated with you application requirements, architecture. Now, the risk needs to be evaluated using the 5*5 Risk assessment Matrix. So, Risk Score = Likelihood * Severity.]*

Technical Risk	Likelihood (1–5)	Severity (1–5)	Risk Score (L×S)	Mitigation Strategy	Testing Approach
e.g., Payment Deducted but order status not updated	*3*	*5*	*15 (High)*	*Implement automatic rollback strategy to handle such failures. [e.g. Database transaction timeout increased to 10s]*	*Use manual/force failure after payment and monitor what gets triggered. [e.g. Sandbox Payment on Stripe]*

Risk Assessment Scoring Criteria:

Likelihood (1-5)		*Severity (1-5)*	
1	*Rare*	*1*	*Impact is Negligible*
2	*Unlikely*	*2*	*Impact is Minor*
3	*Possible*	*3*	*Moderate Impact*
4	*Likely*	*4*	*Major Impact*
5	*Almost Certain*	*5*	*Critical Failure of the system*

Risk Score = Likelihood × Severity

Risk Level Reference:

Risk Score	***Risk Level***	***Recommended Action***
1 – 5	*Low*	*Monitor only. Acceptable risks.*
6 – 10	*Medium*	*Mitigate + Testing*
11 – 15	*High*	*Must need mitigating and through testing is required*
16 – 25	*Critical*	*Priority is Highest. Need extensive level of testing.*

© UMHackathon 2026

Email: umhackathon@um.edu.my

3. Test Environment & Execution Strategy

Explain the testing environment. That means where the testing of your application will take place, how you are going to handle test data and the rules/threshold that you are incorporating to govern your testing phases.

- **Unit Test**
    - *[e.g., Scope: Payment Validation etc.]*
    - *[e.g., Execution: The tests are written using a framework like PyTest. So, execution happens while developing as well as CI pipeline upon push to repo]*
    - *[e.g., Isolation: To focus only on the logic of payment, external dependencies (e.g. Database) are mocked.*
    - *[e.g., Pass Condition: Achieved happy cases, negative cases with proper return format like "Credit Card No. should be 16 Digit" and considered edge cases]*
- **Integration Test**
    - *[e.g., Scope: Order Service to Menu Service.]*
    - *[e.g., Execution: This test can be performed after proper completion of Unit Testing like "Merging two Branches of Order Service and Menu Service"]*
    - *[e.g., Workflow: To ensure the API calls and interactions with databases are working, so real calls are tested without mocking.]*
    - *[e.g., Pass Condition: When integrated components like "menu to order" are working together correctly and showing expected outcome.]*
- **Test Environment (CI/CD Practice):**
    - *[e.g., Local Testing: This testing is manual one on localhost]*
    - *[e.g., Staging/CI: GitHub Actions triggers tests on every push of code.]*
    - *[e.g. CI/CD automated pipeline: So, GitHub Actions does basic tests for branch specific level and then PyTest for PR (Pull requests) to Main branch]*

- **Regression Testing & Pass/Fail Rules:**
    - *[e.g., Execution Phase: This testing tests across all 3 in-scope features each time a new branch is merged to main branch.]*
    - *[e.g., Pass/Fail Condition: Test case pass is only considered when actual outcome perfectly aligns with expected one. If mismatch/threshold not achieved, the test status is considered fail and need to incorporate that in Defect Log]*
    - *[e.g., **Continuation Rule:** While doing testing in a gradual manner, the initial cases must need to meet the threshold to allow continue for further testing]*

- **Test Data Strategy:**
    - *[e.g., Manual: Created 5 Mock Users account with different roles like Customer, Vendors, Rider, Administrators and guest account]*
    - *[e.g., Automated: SQL seed scripts can be used an input for the automated uses of dummy data]*

© UMHackathon 2026

Email: umhackathon@um.edu.my

- **Passing Rate Threshold:**
    - *[e.g., A minimum of 86.2% of all test cases that are executed must pass. For the critical tests, all the tests (100%) need to be passed and there is no exception for that. Because critical failure costs huge in production]*

4. CI/CD Release Thresholds & Automation Gates

This section is mainly for defining metrics & threshold for success criteria. So, if a certain threshold is not met, the pipeline blocks the forward transition.

4.1 Integration Thresholds (Merging to Main)

The primary needs of this threshold to be crucial is because the main branch must be a stable source of truth. So, it needs to be achieved through continuous integration checks.

Checks	***Requirements***	***Project***	***Pass/Failed***
Automatic Build	***Zero Build Error***	***5 Errors***	***Failed***
Unit Tests	***100% Passing Rate***	***100%***	***Passed***
Code Quality	***Zero Linting Error***	***97%***	***Failed***
Test Coverage	***Minimum 81.2%***	***79%***	***Failed***

© UMHackathon 2026

Email: umhackathon@um.edu.my

4.2 Deployment Thresholds (Pushing to Production)

Checks	***Requirements***	***Project***	***Pass/Failed***
Regression Test	***Minimum 90%***	***91%***	***Passed***
AI Output Pass Rate	***Minimum of 80% of documented prompt***	***78%***	***Failed***
Critical Bugs	***Zero P0/01 Bugs***	***2***	***Failed***
API Performance	***Response Time < 800ms***	***450ms***	***Passed***
Security	***API keys are not exposed***	***Clean***	***Passed***

5. Test Case Specifications (Drafts)

This section is for you to draft your test cases. The test cases MUST include at least:

- ***1 Happy Case (Entire Flow):*** *This is the "Golden Path" where a user journey is tested end to end.*
- ***1 Specific Case (Negative/Edge Test):*** *This case should portray a scenario which triggers error in the system and the system handles it with proper handling techniques.*
- **2 Non-Functional (NFR) Tests (Performance/Load)*****:*** *This test consists of non-feature-based tests but architectural tests.*

Test Case ID	**Test Type & Mapped Feature**	**Test Description**	**Test Steps**	**Expected Result**	**Actual Result**
TC-01	***Happy Case (Entire Flow):*** *Food Order*	*Verify whether a user can browse the menu, and then place the order of their choice, complete the payment and receive the confirmation of delivery. So, all the modules are*	*1. Open Website, login with credentials.* *2. Search Restaurant and add food on cart* *3. Proceed to*	*Order should have been placed usefully with payment is also deducted exact amount and notification is received with 5s after*	*Successfully, Order Placed. Payment deducted and receipt is also generated. Vendors receive notification with 1.18s. The rider*

© UMHackathon 2026

Email: umhackathon@um.edu.my

Test Case ID	Test Type & Mapped Feature	Test Description	Test Steps	Expected Result	Actual Result
		communicating with each other correctly.	*Order Confirmation screen* *4. Selecting Payment Method, confirmation of method* *5. Vendors gets notification of order* *6. Rider picks up the order* *7. Users received the delivery properly.*	*confirmation. The rider was assigned and delivered the food by estimated time.*	*was assigned and delivered the correct order in time.* *Status: Passed*
TC-02	***Specific Case (Negative):*** *Invalid Card Details*	*Verify the system does handle error quite optimally and block access when an invalid card is entered without triggering 500 errors.*	*1. Go to the Payment Screen.* *2. Enter card number of below 16 digits.* *3. Clicking on confirm payment*	*Payment is blocked. Showing "Invalid Card Number" error message in the UI. Suggestion message to provide missing digits. Users can retry.*	*Payment is blocked, but an error message is not displayed. App Remains stables. The user is not aware of what happened.* *Status: Failed*
TC-03	***NFR (Performance):*** *API Response Test*	*Payment API remains quite responsive (< 800ms) when concurrent users are using.*	*1. Open Postman, selecting the API Endpoints.* *2. 50 users GET Requests to that endpoint.* *3. Record the average response time.*	*Average response time < 800ms in addition 0% error rate.*	*Average Response time is received as 1128ms. 0% Error Rate.* *Status: Failed*

6. AI Output & Boundary Testing (Drafts)

© UMHackathon 2026

Email: umhackathon@um.edu.my

This section is designed to validate that your GLM integration does produce acceptable output and handles any inputs that are abnormal gracefully.

6.1. **Prompt/Response Test Pairs**

Document at least 3 Prompts/response test pairs. For each of them, do define the acceptance criteria - what a correct, acceptable or a failing response may look like for your use case.

Test ID	Prompt Input	Expected Output (Acceptance Criteria)	Actual Output	Status
AI-01	[e.g. Summarize the 3-paragraph of customer complaint]	[e.g. The output should be only 2-4 sentence, does capture main issue, no fabrication]	Your Test Output	Pass/Fail
AI-02	[e.g. Generate report from a structured Json input]	[e.g. All the JSON fields does reflect accurately and there is no hallucinated values]	Your Test Output	Pass/Fail

6.2. Oversized/Larger Input Test

In this section, do define the maximum input size your system accepts. Do highlight here what happens when the limit is exceeded.

Fields	Details
Maximum Input Size	[e.g. 1500 Token/800 Word Count/ 3 Page Upload]
Input used while testing	[e.g. 9000 Words - 6x of the limit]
Expected Behavior	[e.g. Reject / Chunking into segments]

© UMHackathon 2026

Email: umhackathon@um.edu.my

Actual Behavior	You test Output
Status	Pass/Fail

6.3. Adversarial/Edge Prompt Test

In this section, test minimum of 1 with one prompt that is ambiguous, malformed - document how GLM does respond and how your system is going to handle it.

6.4. Hallucinating Handling

This section is defined for you to describe how your system does detect or mitigate hallucinating response.

Important Note:

Using Agile principle, these testing will take place iteratively over the period of development, not limited to at the end.

© UMHackathon 2026

Email: umhackathon@um.edu.my